



Improving Checkpointing I/O Performances of the Gysela HPC Application

Alexandre Lou-Poueyou, Francieli Boito, Méline Trochon, Luan Teylo

**TECHNICAL
REPORT**

N° XXXX

August 2026

Project-Teams TADaaM

ISSN 0249-0803 ISRN INRIA/RT--XXXX--FR+ENG



Improving Checkpointing I/O Performances of the Gysela HPC Application

Alexandre Lou-Poueyou*, Francieli Boito[†], Méline Trochon[‡],

Luan Teylo[§]

Project-Teams TADaaM

Technical Report n° XXXX — August 2026 — 8 pages

Abstract: In modern High-Performance Computing (HPC) systems, Input/Output (I/O) performance represents a critical bottleneck for large-scale scientific simulations. As computational power scales towards exascale, storage bandwidth has not kept pace, creating a significant compute-to-I/O performance gap. Saving application states through periodic checkpointing is essential for fault tolerance, yet synchronous checkpoint writing halts computation and wastes expensive CPU/GPU hours.

This work focuses on optimizing the checkpointing performance of Gysela, a 5D gyrokinetic simulation code developed by the CEA for nuclear fusion research. Gysela generates massive datasets and relies on periodic checkpointing for execution restartability. Using `gys_io`, a lightweight mini-application replicating Gysela’s parallel 5D I/O patterns, we systematically evaluate three parallel I/O strategies—Single Shared File (SSF), Subfilng, and File Per Process (FPP)—alongside Parallel File System (PFS) striping configurations on clusters.

Key-words: HPC, Parallel I/O, Performance Tuning, Parallel File Systems

* Computer Science Master’s Student at University of Bordeaux – Intern at TADaaM team, Inria Bordeaux

[†] Associate Professor at University of Bordeaux & Permanent Researcher in TADaaM team, Inria Bordeaux

[‡] PhD Student in TADaaM team, Inria Bordeaux

[§] Permanent Researcher in TADaaM team, Inria Bordeaux

RESEARCH CENTRE
Centre Inria de l’Université de Bordeaux

200 avenue de la Vieille Tour
33405 Talence Cedex

Contents

1	Introduction	4
2	Institutional & Project Context	5
3	Background & High-Level Write Strategies	5
3.1	Parallel File Systems and Data Striping	5
3.2	Parallel Write Strategies	6
3.2.1	Single Shared File (SSF)	6
3.2.2	Subfiling	6
3.2.3	File Per Process (FPP)	7
3.3	Software Stack and Tooling	7
4	Experimental Setup	7
4.1	Hardware and Storage Infrastructure	7
4.2	Benchmark Workload & Evaluation Parameters	8
5	Experimental Evaluation and Analysis	8

1 Introduction

In modern High-Performance Computing (HPC) systems, Input/Output (I/O) throughput has become as critical to overall efficiency as computational capability itself. As supercomputing architectures advance toward and beyond the exascale threshold, computational throughput has increased exponentially. However, persistent storage bandwidth has lagged behind, widening the compute-to-I/O performance gap.

While modern processors achieve multi-petaflop or exaflop performance, parallel file systems typically deliver only hundreds of gigabytes to a few terabytes per second in aggregate write bandwidth. This growing imbalance creates a severe bottleneck: high-performance computing tasks spend a non-negligible fraction of their allocated runtime waiting for data transfers to complete. The compute-to-I/O ratio (measured in GB/s per PetaFlop) has degraded by more than an order of magnitude over the past two decades.

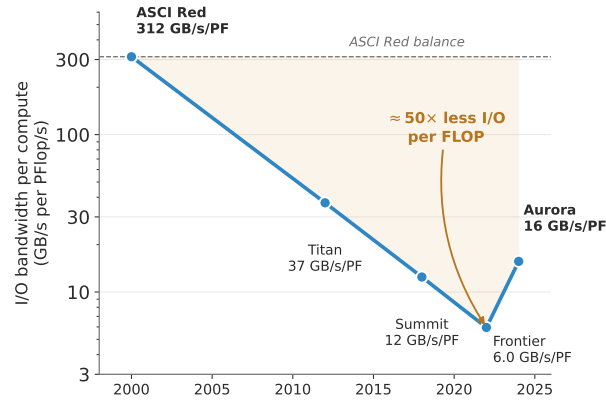


Figure 1: Compute-to-I/O performance gap in Top500 supercomputers (2000–2025). The ratio of storage bandwidth per unit of compute has degraded by more than an order of magnitude.

Large-scale scientific simulations generate massive volumes of data requiring efficient parallel management. On shared Parallel File Systems (PFS), thousands of concurrent processes must coordinate access to shared storage arrays while contending for network link bandwidth and storage controller resources.

A prominent example is Gysela, a 5D gyrokinetic simulation code developed by the CEA (*Commissariat à l'Énergie Atomique et aux Énergies Alternatives*) for nuclear fusion research. Gysela models plasma turbulence in magnetic confinement devices (tokamaks) by solving the 5D Vlasov-Poisson system across three spatial coordinates (minor radius r , poloidal angle θ , toroidal angle φ) and two velocity-space coordinates (parallel velocity $v_{\parallel}$, magnetic moment μ). Understanding and mitigating plasma turbulence is crucial for maximizing thermal confinement in next-generation fusion reactors such as ITER.

Because production Gysela simulations execute across hundreds to thousands of computing nodes over extended periods (days to weeks), they rely on periodic checkpointing for fault tolerance and job restartability. The checkpointing procedure is synchronous: all parallel processes pause numerical integration, aggregate their local state vectors, and flush the 5D distribution function to persistent storage. During this phase, compute resources remain idle. Moreover, in shared production environments, checkpoint write times exhibit high variability—sometimes exceeding median write durations by a factor of 3—due to concurrent I/O contention from uncoordinated tenant jobs.

Optimizing checkpoint I/O performance requires a coordinated approach across two complementary layers:

1. **Parallel File System (PFS) Configuration:** Adjusting low-level layout parameters, such as striping count (number of storage targets) and stripe size.
2. **High-Level I/O Strategies:** Selecting how application data structures are mapped, aggregated, and formatted when dispatched to storage (e.g., Single Shared File, Subfiling, or File Per Process).

To enable rapid, reproducible experimentation without incurring the massive computational cost of full plasma physics simulations, we employ `gys_io`, a dedicated mini-application that faithfully reproduces Gysela’s 5D checkpoint I/O access pattern. In this report, we evaluate the impact of PFS parameters and high-level write strategies, analyze scaling behaviors on the Plafrim benchmark platform, and formulate actionable deployment recommendations for Gysela production runs.

2 Institutional & Project Context

This work was conducted as part of a Master’s research internship at Inria (the French National Institute for Research in Digital Science and Technology) within the TADaaM research team at the Inria Centre at the University of Bordeaux. TADaaM (*Topology-Aware system-scale Data management for high-performance computing*) specializes in designing system software and data management layers that bridge the gap between application-level requirements and complex hardware topologies.

This research contributes directly to the NumPex program, a major French national initiative designed to build the software ecosystem for exascale supercomputing. Specifically, the study falls under the Exa-DOST work package, which focuses on developing advanced I/O optimization strategies, dynamic data management middlewares, and workflow scheduling tools for extreme-scale applications.

The target application, Gysela, is maintained and advanced by researchers at CEA. Collaborative efforts between Inria TADaaM and CEA aim to prepare Gysela’s I/O substrate for upcoming European exascale supercomputers (such as Alice Recoque in France).

3 Background & High-Level Write Strategies

3.1 Parallel File Systems and Data Striping

Traditional file systems rely on single storage controllers, which creates a severe bottleneck when thousands of parallel MPI ranks attempt simultaneous file access. Parallel File Systems (PFS)—such as Lustre, BeeGFS, and IBM Spectrum Scale (GPFS)—bypass this bottleneck by partitioning files across multiple independent storage devices via data striping.

In systems such as BeeGFS and Lustre, storage capacity is divided into logical entities called Object Storage Targets (OSTs). An OST represents a dedicated physical storage unit (e.g., a RAID array or NVMe volume) managed by an Object Storage Server (OSS). Metadata servers (MDS) maintain directory structures and file layouts, while OSTs handle actual data payload transfers.

Data striping partitions a logical file stream into contiguous blocks of size S_{stripe} (Stripe Size). These blocks are distributed in a round-robin sequence across N_{OST} targets (Stripe Count or

OST Count). For example, with $N_{\text{OST}} = 4$ and $S_{\text{stripe}} = 1$ MB, the first megabyte is written to OST 0, the second to OST 1, and so on. Striping allows applications to aggregate the physical bandwidth of multiple storage devices, ideally achieving an N_{OST} -fold increase in overall write throughput.

3.2 Parallel Write Strategies

The layout of output files significantly impacts storage efficiency. In this study, we analyze three primary high-level I/O strategies, implemented via the Parallel Data Interface (PDI) middleware:

3.2.1 Single Shared File (SSF)

In the Single Shared File approach, all MPI processes concurrently write their local data slices into a single unified file on the PFS (Figure 2). Standard libraries (such as HDF5 or MPI-IO) employ two-phase I/O to aggregate non-contiguous memory layouts into large, contiguous file offsets before issuing collective storage calls. While SSF simplifies dataset management by producing a single output file, it can suffer from locking contention on storage targets when rank counts scale up.

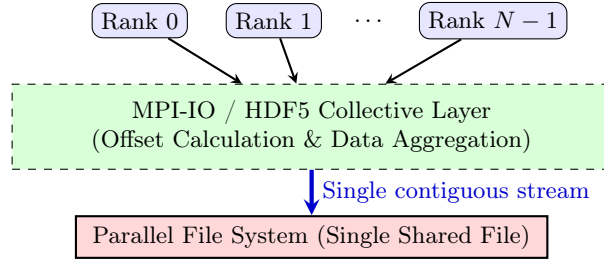


Figure 2: Single Shared File (SSF) strategy overview.

3.2.2 Subfiling

Subfiling acts as an intermediate approach between SSF and individual file writes. MPI processes are partitioned into distinct sub-groups. Within each group, an assigned process—referred to as an I/O Concentrator (IOC) or aggregator—collects data from its peer ranks and writes it to a dedicated subfile (Figure 3).

This reduces file system lock contention compared to SSF, while avoiding the metadata overload associated with generating thousands of individual files. The total number of generated subfiles (N_{subfiles}) is a tunable parameter.

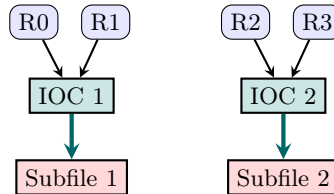


Figure 3: Subfiling strategy overview with two I/O Concentrators (IOCs).

3.2.3 File Per Process (FPP)

In the File Per Process approach, every MPI process creates and writes its own separate file independently (Figure 3.2.3). Because there is no inter-process communication or offset synchronization required, local write throughput can be very high. However, FPP generates N_{ranks} separate files per checkpoint, which can strain metadata servers (MDS) at scale and complicate downstream post-processing or data management.

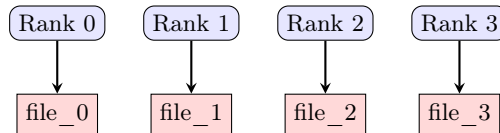


Figure 4: File Per Process (FPP) strategy overview.

3.3 Software Stack and Tooling

To evaluate these strategies, our experimental framework combines three key software components:

- **gys_io Mini-App:** Developed by CEA, *gys_io* is a lightweight benchmark proxy that replicates Gysela’s exact 5D domain decomposition and checkpoint write calls while skipping computational physics routines. It reads Gysela configuration files directly, enabling fast, reproducible I/O benchmarks.
- **Parallel Data Interface (PDI):** PDI decouples application code from underlying I/O libraries. By modifying external YAML configuration files, PDI allows switching between SSF, Subfiling, and FPP backends without recompiling the application binary.
- **IOPS Benchmark Automation:** IOPS (*I/O Performance Search / Suite*) automates parameter sweeps, manages SLURM batch job submission, and extracts execution metrics (such as `time_write` and aggregate write bandwidth).

4 Experimental Setup

4.1 Hardware and Storage Infrastructure

Experiments were performed on the **Plafrim** cluster hosted at Inria Bordeaux. The tests were run on **Bora** compute nodes with the following specification:

- **Nodes:** Dual-socket Intel Xeon Gold 6240 processors (36 physical cores per node, 192 GB RAM per node).
- **Interconnect:** Omni-Path 100 Gb/s fabric.
- **Parallel File System:** BeeGFS parallel file system backed by up to 8 active Object Storage Targets (OSTs).

4.2 Benchmark Workload & Evaluation Parameters

The test configuration used a standard 5D grid resolution matching representative Gysela runs:

$$N_r \times N_\theta \times N_\varphi \times N_{v_\parallel} \times N_\mu = 128 \times 128 \times 64 \times 64 \times 64 \quad (1)$$

This setup yields a single-checkpoint memory footprint of approximately 34.35 GB.

All benchmarks were submitted in exclusive node allocation mode with 8 tasks per node across process scales ranging from 16 MPI ranks (2 nodes) up to 128 MPI ranks (16 nodes). To account for background network and storage noise, each experimental point represents the average of 5 independent executions.

Key measured metrics include:

- **Write Time (T_{write}):** Effective time spent within the PDI write call in seconds.
- **Aggregate Write Bandwidth (BW_{write}):** Defined as total dataset volume divided by write time, expressed in Gbit/s:

$$BW_{\text{write}} = \frac{\text{Data Volume (Bits)}}{T_{\text{write}} \text{ (s)}} \times 10^{-9} \quad (2)$$

5 Experimental Evaluation and Analysis

The Inria logo is a red, stylized script wordmark that reads "Inria". It is positioned inside a white rectangular box with rounded corners, which is set against a light gray background.

RESEARCH CENTRE
Centre Inria de l'Université de Bordeaux

200 avenue de la Vieille Tour
33405 Talence Cedex

Publisher
Inria
Domaine de Voluceau - Rocquencourt
BP 105 - 78153 Le Chesnay Cedex
inria.fr

ISSN 0249-0803